

UC3CND10 - Computer Network Defence

Installation & Usage Guide (validate-adesha01182.sh)

Generated: 2026-02-05 08:44

1. Purpose

This document explains how to install, configure, test, and schedule the validation script for daily SOC-style checks. The script downloads a host-specific control file over HTTPS, verifies its detached signature, validates tool integrity, creates a backup of configured targets, generates a local report, packages outputs into a signed archive, uploads the results over SSH, and performs remote signature verification.

2. Operating requirements checklist

- All script assets operate under /opt/security (K1). /opt should remain read-only except when writing required outputs.
- The -d parameter must be an HTTPS base URL (K2).
- Control files are located at: ///.dat and .dat.gpg (K4-K6).
- The upload host must follow: backup..int.org and SSH runs on port 2069 (K9-K11).
- The userid (-u) selects the SSH key at /opt/security/.id (K10).
- The system must be returned to its pre-run state, especially /opt mount mode (K12).

3. One-time installation steps

Run the following as root (or with sudo) on the target server:

```
# 1) Ensure directories exist
sudo mkdir -p /opt/security/tmp /opt/security/errors /opt/security/bin

# 2) Place the script in /opt/security (recommended) and lock down permissions
sudo cp validate-adesha01182.sh /opt/security/
sudo chmod 700 /opt/security/validate-adesha01182.sh
sudo chown root:root /opt/security/validate-adesha01182.sh

# 3) Confirm required tools exist (these are provided by the course environment)
ls -l /opt/security/bin/validate /opt/security/bin/sectar

# 4) Confirm the SSH identity exists for your upload user
ls -l /opt/security/uploader123.id

# 5) Create a non-interactive passphrase file for signing (S13)
# Put only the passphrase text inside the file (no extra spaces/lines).
sudo mount -o remount,rw /opt
sudo nano /opt/security/sign.pass
sudo chmod 600 /opt/security/sign.pass
sudo chown root:root /opt/security/sign.pass
sudo mount -o remount,ro /opt
```

4. Pre-execution sanity checks

These checks confirm the environment before running the script:

```
# Syntax check
bash -n /opt/security/validate-adesha01182.sh && echo "OK: syntax clean"

# Confirm /opt mount mode (should normally be ro)
```

```
findmnt -no SOURCE,TARGET,FSTYPE,OPTIONS /opt

# Confirm GPG key availability (signing)
gpg --list-secret-keys "tht2023@tht.noroff.no"

# Confirm validate/sectar are executable
ls -l /opt/security/bin/validate /opt/security/bin/sectar
```

5. Script execution and parameters

Execution format:

```
/opt/security/validate-adesha01182.sh -d <BASE_URL> -r <REGION> -u <USERID>
```

Parameter meanings:

Parameter	Meaning
-d	HTTPS base URL used to download the data file and signature (K2).
-r	Region identifier used in the download path and to construct the upload host backup.<region>.int.org
-u	Upload user account name; selects /opt/security/<userid>.id for SSH authentication (K10).

Example:

```
sudo /opt/security/validate-adesha01182.sh -d https://localhost -r eu01 -u uploader123
```

6. Output format and error handling

All script output is plain text written to STDOUT. Fatal errors print a single failure line and exit immediately. Warnings (S8) are printed and execution continues; if any warning occurs, the script bundles the working directory into /opt/security/errors/error-YYYYMMDD.tgz at exit.

Expected success lines include:

```
UPLOAD <serverid>-validate-YYYYMMDD.tgz <size>MB <sha256>
VALIDATE datafile Check for <hostname> YYYYMMDD-HH:MM OK
```

7. Cron automation (S18)

To run daily without interaction, add a root cron entry. Cron captures STDOUT and can email the results.

```
# Edit root crontab
sudo crontab -e
```

```
# Example: run daily at 03:30
30 3 * * * /opt/security/validate-adesha01182.sh -d https://dataserver.int.org/validate
-r eu01 -u uploader123
```

8. Local HTTPS + Apache test lab (optional, for S3-S6 testing)

If you need to simulate the download server locally, configure Apache with HTTPS and serve the expected path structure ///.dat and .dat.gpg under the DocumentRoot.

```
# Quick check the local web server responds with a valid cert
curl -I https://localhost/
```

```
# Confirm the two files are reachable (adjust REGION/TODAY/SERVERID as needed)
curl -sS "https://localhost/eu01/$(date +%Y%m%d)/<serverid>.dat" | head
curl -sS "https://localhost/eu01/$(date +%Y%m%d)/<serverid>.dat.gpg" | head
```

9. Packaging for submission

The final submission archive must contain three files at the root: the script, the detached signature for the script, and this PDF guide. Use the helper script `make_submission_tgz.sh` to generate the .sig and package the .tgz.

```
# In the folder where you have the script and PDF:
chmod 700 make_submission_tgz.sh
sudo bash ./make_submission_tgz.sh validate-adesha01182.sh adesha01182.pdf
```

10. Post-run sanity checks

```
# Confirm /opt mount options are restored (K12)
findmnt -no OPTIONS /opt

# Confirm outputs exist for today
TODAY="$(date +%Y%m%d)"
ls -lh /opt/security/tmp/$TODAY/*validate-*.tgz
/opt/security/tmp/$TODAY/*validate-*.tgz.sig 2>/dev/null
```